

The Device Model, and What a Narrow Interface Survives

UM-NATOS-031

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-031	1.1 · 2026-08-18	**Shipped; six devices and bulk transfer verified on hardware**	Internal

1. Abstract

UM-NATOS-007 §2.1 named one structural item missing from the roadmap, and book chapter 31 restated it seventeen reports later:

Every driver above is reachable only from the kernel. The VM has twelve syscalls, all hardcoded, and no device model — so an application cannot read the light sensor, scan the I²C bus or receive a keypress. Each new peripheral has meant a kernel edit plus a hand-written syscall, which was tolerable at two and is the obvious next piece of **architecture** rather than more drivers.

It is done. `sys device` is the **last** hand-written syscall; everything after it is a table entry. Five devices exist, all verified against hardware, and the list of things an application cannot do has gone from eight items to four.

The interesting part is not the dispatcher. It is what happened to a deliberately narrow interface when four more devices were pushed through it: three fitted without complaint, one forced a change, and the change it forced was the right one. §4 keeps that record, because "we designed it and it worked" is worth nothing without the entry that did not fit.

Three defects were introduced and caught during the work, all three by the same mechanism as the last two reports: **something reported success while doing the wrong thing**. §6 has them.

2. The harness came first

Chapter 31 named a second gap in the same breath:

Nothing enforces that a future one will [check its arguments], and there is no shared harness that would catch an unchecked length in a new service.

Twelve services was tolerable. A device model turns twelve into any number, and an ad-hoc check per service does not survive that. So `kernel/vmarg.c` landed **before** the model, and the model had nothing to invent.

Nothing in it is new policy. Every rule is lifted from a site that already got it right; the point is that there is now one copy.

rule	why	taken from
<code>len <= size - off</code>	<code>off + len</code> wraps, and a hostile length aims at that	<code>vm_in_bounds()</code>
bound count before multiplying	a 65536×1 image passes a byte check then blits what follows	<code>SYS BLIT</code>
copy strings, never lend a pointer	the arena belongs to a program that runs again the instant the call returns	<code>copy_string()</code>

Failure records the fault on the vm *before* returning 0, so a caller's whole error path is `return 1;`. A harness that could only refuse without diagnosing would trade a bounds bug for a silent one — `vm_raise()` is exported for exactly this, and `vmarg.c` lives outside `vm.c` because the device model shares it and does not live there either.

Both existing sites were ported onto it, which is the only real proof the shape is right. `SYS BLIT` became `vmarg_items(off, w*h, 2, DISP_W*DISP_H, 2)` — the call the function was generalised *from*, so it had better fit.

2.1 The reject counter is load-bearing

`vmargtest` drives known-bad arguments through the harness and requires each to be refused **with the right fault code**: 12/12 pass, `checks=13 rejects=7` per run, including the wrapping case the offset-domain rule exists for.

The rejection count is printed because a harness that has never rejected anything and a harness that is never reached look identical from outside, and this kernel has been caught by that shape three times — an audio self-check that could not fail by construction (UM-NATOS-027), a viewport counter that only moved in one drawing mode (UM-NATOS-028), and a DMA timeout counter read as zero from a build that already contained the fix (UM-NATOS-030 §5.4). If the number stops moving, the command prints `ZERO REJECTS`, the test is inert.

3. The interface

```

sys device
  r0 = op
  0 COUNT                -> r0 = how many
  1 NAME  r1=id r2=off r3=max -> name written into the caller's arena
  2 READ  r1=id r2=chan      -> r0 = ok, r1 = value
  3 WRITE r1=id r2=chan r3=value -> r0 = ok
  4 INFO  r1=id              -> r0 = ok, r1 = channels, r2 = flags

```

One word in, one word out, on a numbered channel. That is what every peripheral this board actually has needs — a light level, a tone, a key, a byte on a bus — and a richer interface would put a program-supplied buffer at *every* entry, which is a far larger surface to get right.

Chapter 31's four properties all hold:

- **Offset domain.** The one buffer in the interface (`NAME`) goes through `vmarg_store`.

- **Bound then multiply.** Nothing here multiplies a program-supplied count; `vmarg_items` is ready when something does.
- **Copy, do not lend.** `vmarg_store` copies *into* the arena. No writable arena pointer is produced anywhere — the mirror of the borrowed-string rule, and it fails the same way if broken.
- **The quantum.** `DEV_F_SLOW` sets `yield_now`, so a call costing milliseconds is not charged to the instruction quantum.

Refusal is not a fault. A bad channel, an unsupported direction or an absent device returns 0 and leaves the program running. Asking a device something it cannot answer is legal, and a program that cannot enumerate without dying cannot enumerate. Faults stay what they have always been: reaching outside the arena.

A driver never sees the vm and never sees an arena. It receives validated scalars and returns success or refusal. Channel bounds are checked in the table *as well as* in the driver, so a driver that forgets cannot become a hole — the same reasoning as `SYS BLIT` re-deriving its rectangle after clipping.

4. What five devices did to a narrow interface

The two starting entries were **ports** of drivers that already worked from the kernel, chosen on purpose: an abstraction proved only by code written to fit it has been proved of nothing. Neither changed to fit the table.

#	device	fitted?	what it cost
1	<code>light</code>	yes	nothing — ADC1 ch6, 8-sample average
2	<code>beep</code>	yes	nothing — packs <code>(hz << 16) \ ticks</code> into one word
3	<code>store</code>	no	<code>device_t</code> grew a <code>caller</code> argument
4	<code>i2c</code>	yes	nothing
5	<code>keys</code>	yes	nothing — but exposed <code>DEV_F_CONSUME</code> (§6.3)

4.1 The entry that did not fit

`store` gives an application four persistent words. They had to be **banked by caller**: everything else an application owns here is confined to it — its arena, its viewport, its mailbox — and persistence with one shared pool would be the single place a program could read what another program wrote.

`device_t` could not express that. It grew a `caller` argument, most drivers ignore it and say so, and the point is that a driver needing it can have it without inventing its own way to find out — precisely the per-service improvisation the model exists to stop.

The caller comes from `vm->app_id`, never from a register. A program naming its own bank is the same shape of mistake as trusting an offset. Bank `APP_MAX` belongs to the kernel and the shell, so a diagnostic at the prompt cannot land on an application's saved state either.

Two flash-endurance decisions came with it. A write lands in the in-RAM record and reaches flash on the next periodic save, because an erase per write costs tens of milliseconds with interrupts masked and would spend a sector rated for a hundred thousand cycles in an afternoon. And **writing an unchanged value does not**

mark the record dirty — otherwise a program looping on a constant write forces an erase a minute, forever, while believing it is doing nothing.

4.2 What did not fit, and the operation it earned

`i2c_read()` and `i2c_write()` take **buffers**. Revision 1.0 said a fifth operation for them should be driven by a device that needed it rather than guessed at. It was, in the same session:

```
DEV_OP_XFER_OUT  r1=id r2=chan r3=arena offset r4=length  arena -> device
DEV_OP_XFER_IN   same                                device -> arena
```

Two operations rather than one with a direction flag. A caller passing the wrong direction to a combined operation gets a plausible-looking transfer the wrong way, and the two have genuinely different consequences for the arena: OUT only reads it, IN writes into it.

No arena pointer ever reaches a driver, in either direction. Every byte crosses a 64-byte kernel bounce buffer. That is deliberately stricter than `SYS_BLIT`, which lends `display_blit()` a const view of the arena — defensible for one known function reviewed beside its check, and not defensible for a device table, which is the *extensible* surface. Every future driver author would otherwise have to be trusted with the lifetime of a pointer they were handed. Sixty-four bytes removes the question.

Two rules came out of writing it:

- **Refuse, do not clamp.** A caller asking for more than the buffer holds is refused. A silently shortened transfer is a program being lied to about how much it moved.
- **Validate the destination before the device acts.** On the IN path the arena offset is checked before anything is asked to produce bytes, because a device that has already had its side effect cannot be un-asked.

4.3 `echo`, and why a loopback is not scaffolding

The only transfer-capable peripheral is the I²C bus. On a bare board with nothing in the expansion header, the whole of `DEV_OP_XFER` would have shipped having exercised **nothing but its refusal path** — precisely the self-test-that-cannot-fail shape §2.1 and UM-NATOS-027 catalogue.

So the table gained `echo`: a loopback that always answers, and can therefore prove the round trip. It refuses to return more bytes than it was given, because zero-filling the shortfall would let a round trip pass against a device that had lost half the data.

It is the seventh entry in spirit and the sixth in the table, and it fitted without changing anything.

5. Verified on hardware

Nothing below is inferred.

id	name	chans	flags	ch0
0	light	1	r-s-	399
1	beep	1	-ws-	-
2	store	5	rws-	1234
3	i2c	128	r-s-	refused
4	keys	2	r--c	(consumes)

test	result
light from shell and from a program	live ADC, 623..880 across a shadow
beep 880 15	audible; the table's write counter moves
store: write 1234, commit, full reset , read	1234 — survives power loss
store: app writes 777 to slot 0; shell reads slot 0	1234 — banks isolated
i2c channel 0 (a reserved address)	refused, no fault
i2cscan over an empty header	reports nothing, honestly
keys after typing on the panel	[a][a][a][b][c] , 5 pending
dev 0 5 (bad channel)	refused, program survives
vmargtest	12/12, rejects=7
devw 5 0 222 173 190 then devr 5 0 3	de ad be — round trip
devr 5 0 8 (more than was written)	refused
devw 3 80 1 (empty I ² C bus)	refused, no fault
transfer from a program	app_dev sends 4 bytes from its arena, reads them back to a <i>different</i> offset, compares equal

The last row is the one that counts. The shell calls `device_xfer_*` directly and never touches `vmarg_span` or `vmarg_store`, so only a real application exercises the arena handling — the same "diagnostic uses a different path" trap `beep` fell into (§7 rule 6), avoided here by testing what a program does.

`tools/app_dev.vasm` is the end-to-end proof: the first program in this kernel's life to reach a peripheral. It enumerates the table, has the kernel write each name into its own arena, claims a persistent slot, takes sixteen readings and exits. **No kernel edit was needed to write it.**

5.1 A note on [a][a][a][b][c]

`abc` was typed and `aaabc` was delivered. That is correct. `KEYS[0][1]` is "abc2" — a, b and c share one key, phone-style, and `CYCLE_TICKS` is 80 ticks (~760 ms at the measured rate). Three slow taps expire three separate cycles and produce three `a` s.

The queue reported what actually entered the line, not what the user intended, which is the behaviour wanted from a device. The typing experience is a keypad question and predates this work.

6. Three defects, all of which reported success

6.1 A jump that landed past its setup code

`app_dev` 's enumeration loop ended `brz r1, sensing`, which jumped clean over the slot-claim block **and** over `ldi r14, 16`. The counter was still 0 from `vm_init`, the read loop exited on its first test, and the program printed the device table, announced sixteen readings, and reported a light level of zero without ever having read anything.

Nothing diagnosed it because nothing was wrong: every syscall it made succeeded, and the two it skipped were skipped silently. **A jump that lands past setup code is invisible to a machine that cannot know what the code was for.**

6.2 A device id read back from the register it had just overwritten

`DEV_OP_READ` wrote the result into `r1` and then asked `device_is_slow(vm->reg[1])` — testing whether the *light level* was a slow device. Caught before flashing, by reading the code rather than by any test, which is worth admitting: nothing in the system would have complained.

6.3 A diagnostic that ate what it was reporting

The `dev` listing samples channel 0 of every readable device to show something useful. For `keys`, channel 0 **pops a keypress**. Merely listing the table consumed a character.

`DEV_F_CONSUME` now marks devices whose read changes state, and anything enumerating skips them and prints `(consumes)`. A diagnostic that alters what it reports is worse than one that reports nothing — the same lesson as UM-NATOS-030 in a new place, found the same day.

7. Standing rules earned here

1. **Ports prove an abstraction; purpose-built code does not.** `light` and `beep` were existing drivers moved unchanged. If they had needed edits, the shape was wrong.
 2. **The entry that does not fit is the useful one.** Three fitting was encouraging. `store` failing, forcing exactly one well-motivated change, and the next two fitting again is evidence.
 3. **Identity comes from the kernel, never from the caller.** A program naming its own bank is a program supplying its own offset.
 4. **Refusal and zero must be distinguishable.** An empty key queue refuses, because zero is a legitimate character and one word cannot mean both.
 5. **Mark what a read costs, and mark what it destroys.** `DEV_F_SLOW` protects the renderer; `DEV_F_CONSUME` protects the evidence.
 6. **A diagnostic must use the path an application uses.** `beep` called `audio_beep()` directly and was reporting on a route no program can take.
-

8. What is left

Applications still cannot: **read the SD card** or **use the network**. Both are now a table entry plus a driver rather than architecture — which is the whole point of the exercise.

Also open, unchanged by this work: MISO reads all zeros so the panel cannot be read back (UM-NATOS-030 §7); the phantom touches at ~380 s are real, reproducible and unexplained; and transmit still does not reach the air.

Twelve services, and then any number.